



Evaluación #2
PATRONES DE DISEÑO ARQUITECTÓNICO

David Berrío Martínez
Camilo José Ardila Restrepo
Miguel Alejandro Martínez Rueda

Escuela de Ingenierías, Universidad Pontificia Bolivariana

Arquitectura de Software

Docente: César Augusto López Gallego

06 de Septiembre de 2026

Contenido

RETO 2 — ACTIVIDAD 1	3
Análisis de puntos de dolor de la arquitectura actual	3
RETO 2 — ACTIVIDAD 2	5
Entregable 2.1 — Tabla de decisión de patrones	5
Nota sobre la aplicación de Strategy en P-02	8
Nota sobre Factory Method en P-01 (versión final, tras dos correcciones de rumbo)	8
Nota sobre Strategy en P-03.....	8
Nota sobre Observer en P-08 (condición cumplida, y el patrón creció).....	8
Nota sobre State en P-09.....	8
Entregable 2.2 — Bitácora de decisiones frente a la IA.....	9
RETO 2 — ACTIVIDAD 4	9
Entregable 4.1 — Matriz de verificación	10
Factory Method — P-01 (medicamento).....	11
Strategy — P-02 (regla de puntos)	11
Strategy — P-03 (descuento).....	12
Observer — P-08 (notificación y orquestación de venta).....	12
State — P-09 (ciclo de vida de la venta)	12
Entregable 4.2 — Evidencia de preservación de comportamiento (resumen)	13
RETO 2 — ACTIVIDAD 5	13
Análisis de Riesgos	13
Entregable 5.1 — Registro de riesgos.....	13
Vista para el negocio	14
Vista para el equipo de desarrollo.....	15

RETO 2 — ACTIVIDAD 1

Análisis de puntos de dolor de la arquitectura actual

ID	Dónde (archivo / clase)	Qué lo hace rígido o caro	Cuánto cuesta hoy	Prioridad y origen
P-01	Clases/MedicamentoCapsula.cs, Clases/MedicamentoLiquido.cs, Clases/ProductoRequest.cs:15-22, Factories/FabricaProducto.cs:14-36, Repositorios/RepositorioProductos.cs:60-66, AppFarmaciaConsola/Program.cs:375-387, 434, 458, 501	La presentación (cápsula con relleno, líquido con envase) se modela con una subclase por tipo, y esa decisión se repite en cuatro sitios (lista blanca, dos switch, un if con typeof). Una presentación nueva obliga a tocar los cuatro. Los atributos que la justifican (Relleno, Envase, Mililitros) no los usa nadie más.	4 clases a modificar (ProductoRequest, FabricaProducto, RepositorioProductos, Program) y 4 archivos a abrir, más 1 clase nueva por cada presentación que se agregue.	Alta — Propio
P-02	Clases/Cliente.cs:28-33, Servicios/ServicioPuntos.cs:21-30, AppFarmaciaConsola/Program.cs:665-676	La regla de puntos es una suma fija escrita dentro de Cliente; no hay dónde colgar una regla distinta, y hoy ni siquiera existe una regla real — el operador teclea la cantidad a mano. Cambiarla obliga a tocar la entidad, el servicio y el menú a la vez.	3 clases a modificar (Cliente, ServicioPuntos, Program) y 3 archivos a abrir, más una modificación adicional en los mismos tres por cada forma nueva de acumulación.	Alta — Propio
P-03	Interfaces/IDescuento.cs:11, Servicios/ServicioDescuento.cs:15, Clases/Cliente.cs, Repositorios/RepositorioClientes.cs	El contrato de descuento (CalcularDescuento(decimal precio)) no recibe al cliente, así que un descuento que dependa de quién compra no se puede expresar. El único descuento existente es un 10 % quemado en el código, y el cliente no tiene ningún dato de afiliación.	4 clases a modificar (IDescuento, ServicioDescuento, Cliente, RepositorioClientes) y 4 archivos a abrir, más 1 clase nueva por cada combinación de entidad y beneficio: con las cinco entidades y los dos beneficios del Anexo B, hasta 10.	Alta — Propio
P-04	Servicios/ServicioFacturacion.cs:13-19, Servicios/ServicioVenta.cs:31-53, AppFarmaciaConsola/Program.cs:350	El único cálculo de dinero es $\text{precio} \times \text{cantidad}$ sin ningún punto de inserción; peor, la venta ni lo usa (RegistrarVenta no calcula importe) y CalcularTotal solo se invoca al mostrar el historial. No hay momento en el flujo donde decidir cuánto se cobra.	3 clases a modificar (ServicioFacturacion, ServicioVenta, Program) y 3 archivos a abrir.	Alta — Asistido

ID	Dónde (archivo / clase)	Qué lo hace rígido o caro	Cuánto cuesta hoy	Prioridad y origen
P-05	Factories/FabricaProducto.cs:14-36, Repositorios/RepositorioProductos.cs:58-76, Clases/ProductoRequest.cs:15-22, AppFarmaciaConsola/Program.cs:434-516	La primera columna de productos.txt es la única fuente de qué se construye, interpretada en tres sitios. Al resolver P-01 esa decisión queda sin sitio asignado; cuando entren categorías realmente distintas (cosmético, comestible) volverá a repartirse entre el repositorio, la solicitud y el menú.	4 clases a modificar (ProductoRequest, FabricaProducto, RepositorioProductos, Program) y 4 archivos a abrir, más 1 clase nueva por cada categoría de producto.	Media — Asistido
P-06	Interfaces/IFacturable.cs:14, Clases/Producto.cs:27-33, 72-75, Clases/Servicio.cs:50-53, Servicios/ServicioFacturacion.cs:13-19	El precio es un campo fijo: ObtenerPrecio() está declarado como el lugar para variarlo, pero no es sobrescribible y las dos implementaciones devuelven el campo tal cual. Una promoción sobre un producto puntual solo se puede hacer modificando la clase raíz o el precio permanente.	3 clases a modificar (Producto, Servicio, ServicioFacturacion) y 3 archivos a abrir, más 1 clase nueva por cada tipo de promoción.	Media — Propio
P-07	AppFarmaciaConsola/Program.cs:251-703	El menú es un switch de once casos que ocupa 452 de 707 líneas; una opción nueva agrega otro caso. No se interviene: son ramas de captura de entrada, no de negocio, y ninguna solicitud del Anexo B agrega opciones.	1 archivo, y 1 caso nuevo por cada opción que se agregue — solo si se decidiera intervenir	No se interviene — Propio
P-08	BibFarmacia/Interfaces/IServicioNotificacion.cs, BibFarmacia/Servicios/ServicioNotificacion.cs, cableado de eventos en AppFarmaciaConsola/Program.cs:91-136	Existe IServicioNotificacion/ServicioNotificacion sin un solo consumidor; las notificaciones reales van por otro camino (las clases Evento*, cableadas directo a consola). Dos mecanismos para lo mismo, sin límite claro entre evento de dominio y canal de salida.	Para conectar un canal de notificación al flujo de puntos o ventas: 1 clase de canal nueva y 2 clases existentes modificadas (ServicioNotificacion y Program), 3 archivos como mínimo. Si además se amplía la interfaz para soportar destinatario, canal y tipo de evento, se suma IServicioNotificacion, totalizando 4 archivos.	Media — Propio
P-09	AppFarmaciaConsola/Program.cs (case 9, versión previa a Reto 2), Clases/Producto.cs (DescontarStock)	Una venta era un bloque imperativo sin representación de en qué paso iba. Si un paso fallaba a mitad de camino (vender más unidades de las que hay en stock), no existía ningún estado de "venta fallida" que capturar — la excepción tumbaba la aplicación completa.	1 archivo a modificar (Program.cs) para envolver el flujo en manejo de errores, y ningún lugar donde declarar el resultado parcial de una venta fallida — el costo real no es de líneas, es la ausencia total	Alta — Propio

ID	Dónde (archivo / clase)	Qué lo hace rígido o caro	Cuánto cuesta hoy	Prioridad y origen
			de un concepto de "venta en progreso" en el diseño.	

RETO 2 — ACTIVIDAD 2

Entregable 2.1 — Tabla de decisión de patrones

Patrón evaluado	Familia	Punto de dolor que atacaría	Qué gana y qué cuesta	Decisión	Por qué
Strategy	Comportamiento	P-02 — La única forma de que un cliente gane puntos es una suma fija escrita dentro de la propia entidad	Gana: la regla de acumulación deja de estar fija dentro de Cliente Cuesta: una interfaz y una clase por cada regla, y la regla queda fija durante toda la sesión, de modo que cambiar la promoción a media jornada obliga a reiniciar la aplicación.	Adoptado	Hay una operación con varias formas posibles y ninguna razón para privilegiar una. Hoy la única forma es la suma fija de Cliente.cs:28-33 . La regla por defecto reproduce el comportamiento actual, así que la salida no cambia; el administrador inyecta la promoción del día en Program.cs:62-63 , igual que ya se hace con rellenos y envases.
Decorator	Estructural	P-02 — Mismo punto: no hay dónde vivan reglas de acumulación distintas	Gana: permitiría apilar reglas — una base más una promoción. Cuesta: una capa de envoltura permanente y un recorrido más difícil de depurar.	Descartado	Decorator combina reglas, no elige entre ellas, y hoy no existe el escenario de combinarlas
State	Comportamiento	P-02 — Mismo punto, visto como niveles de cliente	Gana: modelaría niveles de cliente en los que acumular puntos cambia el nivel y el nivel cambia la forma de acumular. Cuesta: una clase por nivel más la lógica de transición, y obligaría a agregar a Cliente un atributo de etapa que hoy no existe.	Descartado	State exige que el objeto cambie de comportamiento por su propio ciclo de vida. Aquí no hay ciclo de vida: un cliente no transita entre modos de acumulación y Cliente no tiene ningún atributo de etapa. Adoptarlo exigiría inventar niveles que ningún requisito menciona.
Strategy	Comportamiento	P-01 — La presentación de un medicamento (cápsula con relleno, líquido con envase) está modelada	Gana: un único punto donde ambas presentaciones se resolverían por polimorfismo en vez de por subclase. Cuesta:	Descartado	Strategy exige un contrato común entre las implementaciones intercambiables. IRelleno e IEnvase no comparten métodos: no

Patrón evaluado	Familia	Punto de dolor que atacaría	Qué gana y qué cuesta	Decisión	Por qué
		creando una subclase por presentación.	forzar que IRelleño e IEnvase compartan un contrato.		hay una sola "estrategia de presentación"
Decorator	Estructural	P-01 — Mismo punto: la presentación se modela como subclase en vez de como composición	Gana: construir un Medicamento base y envolverlo con la presentación. Cuesta: dos decoradores, uno por presentación.	Descartado	Medicamento es concreta, así que envolverla es posible, pero Decorator combina comportamientos que coexisten y un medicamento nunca es cápsula y líquido a la vez
Factory Method	Creacional	P-01 — La decisión de qué subtipo construir estaba repetida en ProductoRequest, FabricaProducto, RepositorioProductos y Program.cs	Gana: el switch desaparece por completo de RepositorioProductos; un creador por presentación queda seleccionado por un registro en el composition root. Cuesta: una interfaz y una clase creadora por presentación, más el mecanismo de registro.	Adoptado (tras dos vueltas)	El equipo pasó por tres versiones: Factory Method puro (funcionó, pero se retiró porque el eje de extensión real, no pasa por este switch); la versión final, que combina las dos: CreadorCapsula/CreadorLiquido implementan Factory Method pero delegan IFabricaMedicamentos, conservando la fábrica de sobrecargas.
Command	Comportamiento	P-03 — El contrato de descuento no puede expresar de quién depende un descuento	Gana: poder registrar o deshacer la aplicación de un descuento como operación independiente. Cuesta: una clase de comando por cada tipo de descuento, más la infraestructura para invocarlos.	Descartado	Command encapsula una acción para posponerla, encolarla o deshacerla; su valor está en el historial de ejecución, no en variar un resultado.
Strategy	Comportamiento	P-03 — Mismo punto: el descuento actual es un 10 % quemado, sin manera de expresar de quién depende	Gana: cada convenio pasa a ser una implementación intercambiable de un mismo contrato. Cuesta: la firma actual, CalcularDescuento(decimal precio), no alcanza; hay que ampliarla	Adoptado	Strategy. No se resuelve con sobrecarga de métodos (se decide en compilación); se resuelve sustituyendo en tiempo de ejecución qué IDescuento tiene inyectado cada cliente.
Bridge	Estructural	P-08 — Hay dos mecanismos de notificación sin límite claro entre evento de dominio y canal de salida	Gana: separar en dos jerarquías independientes el tipo de evento (stock, vencimiento, puntos, movimiento) Cuesta: mantener una segunda, cuando hoy solo existe uno, la consola.	Descartado	Bridge se justifica cuando dos ejes de variación crecen por separado. Aquí solo varía el tipo de evento.
Composite	Estructural	P-08 — Mismo punto: los dos mecanismos de	Gana: tratar una notificación individual y un grupo de notificaciones de forma	Descartado	Composite resuelve estructuras parte-todo. No hay ninguna notificación

Patrón evaluado	Familia	Punto de dolor que atacaría	Qué gana y qué cuesta	Decisión	Por qué
		notificación conviven sin límite claro	uniforme. Cuesta: modelar una jerarquía todo.		compuesta de otras ni ningún escenario que pida agrupar avisos.
Observer	Comportamiento	P-08 — El servicio de notificación no lo usa nadie; las notificaciones reales están cableadas directo a la consola	Gana: un límite claro entre qué pasó en el dominio, quién reacciona a eso, sin que quien dispara el evento conozca a quien lo escucha. Cuesta: conectar <code>IServicioNotificacion</code> y <code>ServicioNotificacion</code> .	Adoptado	Las cuatro clases <code>Evento*</code> ya son <code>Observer</code> aplicado a medias, con lambdas de consola como únicos suscriptores. Formalizar ese mecanismo, en vez de introducir uno nuevo, es la opción de menor riesgo: no reemplaza infraestructura que funciona, solo le da el límite que le falta entre evento y canal.
Enum <code>`EstadoVenta`</code> + condicionales	— (no es un patrón)	P-09 — Una venta no tiene representación de en qué punto del proceso va; una falla a mitad de camino es un cuelgue sin control	Gana: una forma rápida de marcar en qué paso va una venta. Cuesta: cada uno de los cuatro servicios que reacciona al evento de venta necesitaría su propio <code>if/switch</code> sobre el mismo enum.	Descartado	Fue la primera propuesta, sugerida por la IA. Se descartó por ser el error típico de mover el condicional en vez de eliminarlo.
Chain of Responsibility	Comportamiento	P-09 — Mismo punto: los cuatro pasos de una venta dependen uno del otro y deben ejecutarse en orden	Gana: una cadena explícita de manejadores (Facturación, Inventario, Movimiento, Puntos) Cuesta: resolvería el orden de ejecución, pero no la pregunta que de verdad importa: si una operación es válida dada la historia de la venta.	Descartado	Resolvería el orden, pero no si una operación es válida dada la historia de la venta. Con <code>Observer</code> ya desacoplando los cuatro pasos, ninguna cadena impide que un servicio procese un contexto fuera de secuencia; un objeto que sabe en qué estado está, sí.
State	Comportamiento	P-09 — Una venta no tiene representación de en qué punto del proceso va; una falla a mitad de camino es un cuelgue sin control	Gana: <code>ContextoVenta</code> sabe en todo momento en qué estado está (Pendiente, Facturada, Procesada, Confirmada, Fallida) y cada estado concreto decide qué transiciones acepta; una transición inválida lanza una excepción controlada en vez de dejar el sistema en un punto intermedio sin nombre. Cuesta: cinco clases de estado más la interfaz, y quien agregue un paso nuevo al flujo de venta tiene que	Adoptado	Los cuatro pasos de una venta reaccionan a un evento compartido (<code>Observer</code> , <code>P-08</code>) y nada por sí solo garantiza el orden. <code>State</code> cierra esa puerta: cada estado concreto solo permite su transición válida, y cualquier otra hereda el <code>throw</code> de <code>EstadoVentaBase</code> . Verificado: vender más unidades de las que hay en stock ya no tumba la aplicación, produce un mensaje controlado.

Patrón evaluado	Familia	Punto de dolor que atacaría	Qué gana y qué cuesta	Decisión	Por qué
			decidir desde qué estado se puede llegar a él.		

Nota sobre la aplicación de Strategy en P-02

La regla se inyecta en el punto de ensamblaje, **no se selecciona desde el menú**.

Nota sobre Factory Method en P-01 (versión final, tras dos correcciones de rumbo)

La versión final combina: CreadorCapsula y CreadorLiquido implementan ICreadorMedicamento (Factory Method, sin switch en RepositorioProductos), pero cada uno envuelve IFabricaMedicamentos en vez de construir el medicamento directamente. Eso conserva la fábrica de sobrecargas para el único caso donde de verdad aporta —el alta interactiva, donde el administrador elige el relleno o el envase en vivo—, y evita duplicar la lógica de construcción entre las dos rutas de entrada (archivo y consola). Es la solución que el argumento de alcance de SC-1 pedía, sin pagar el costo de DIP que la versión intermedia había reintroducido.

Nota sobre Strategy en P-03

Strategy no cerraba el punto de dolor por sí solo, porque el contrato `CalcularDescuento(decimal precio)` no recibe al cliente. Eso se resolvió sin tocar el contrato: Cliente carga la referencia a su propia estrategia (`Cliente.Descuento`), y `ServicioFacturacion.ProcesarVentaSolicitada` la invoca en cada venta real (`contexto.Cliente.Descuento.CalcularDescuento(subtotal)`), con una validación adicional de que el resultado esté entre cero y el subtotal. `DescuentoPorcentual` además pasó a validar su propio porcentaje entre 0 y 1 en el constructor, cerrando la advertencia de sustitución que quedaba abierta cuando este documento se escribió por primera vez.

Nota sobre Observer en P-08 (condición cumplida, y el patrón creció)

La condición que se dejó escrita era que `IServicioNotificacion` tuviera un consumidor real, no solo documentado. Se cumplió: `NotificacionArchivo` está suscrita a la venta y cada venta deja un registro en `notificaciones.log`. Pero el patrón terminó cubriendo más de lo previsto: `EventoVenta` no solo notifica hacia afuera, es el mecanismo que conecta `ServicioFacturacion`, `ServicioInventario`, `ServicioMovimiento` y `ServicioPuntos` entre sí, cada uno reaccionando a la señal del anterior y disparando la siguiente. Ese uso ampliado de Observer es lo que hizo necesario State (P-09): al desacoplar los cuatro pasos en observadores independientes, dejó de haber una sola función que garantizara el orden, y State es lo que impide que se ejecuten fuera de secuencia.

Nota sobre State en P-09

State se evalúa contra un punto de dolor que no viene del código heredado del Reto 1 sino de una consecuencia real de decisiones ya tomadas en este mismo reto: conectar P-02, P-03, P-04 y P-08 en un flujo de venta único, mediante Observer, dejó cuatro clases reaccionando de forma independiente a un mismo objeto compartido (`ContextoVenta`), sin nada que impidiera que se ejecutaran fuera de orden. El punto de dolor original —una venta sin representación de su propio

progreso, donde una falla a mitad de camino era un cuelgue sin control— sí es del Reto 1: está documentado como comportamiento verificado del sistema antes de este trabajo (Producto.DescontarStock lanzando una excepción no capturada en ningún punto de Program.cs).

Entregable 2.2 — Bitácora de decisiones frente a la IA

ID	Qué consultaron	Qué propuso la herramienta	Qué hicieron	Argumento propio y evidencia
B-01	Qué patrón resuelve que la única forma de acumular puntos sea una suma fija dentro de Cliente, sin lugar para una regla distinta (P-02)	Se pidió contrastar Strategy con otras alternativas antes de fijarlo	Fue idea nuestra	“Strategy nos permite escoger diferentes formas de acumular puntos sin romper OCP... Decorator es llevar el código al siguiente nivel, no es necesario; State se descarta por completo, no necesitamos estados, ninguna clase depende de otra.” Strategy adoptado; Decorator y State descartados de una vez.
B-02	Si Decorator servía para la presentación de un medicamento —cápsula con relleno, líquido con envase— (P-01)	Se asumió inicialmente que Medicamento era abstracta, lo que hacía inviable envolverla en un segundo nivel de herencia	Corregimos	Al verificar el código (Clases/Medicamento.cs:9), Medicamento es concreta, no abstracta. El descarte de Decorator se mantuvo, pero por la razón correcta: un medicamento nunca es cápsula y líquido a la vez, es una elección excluyente, no una composición.
B-03	Qué patrón crea cada presentación de medicamento con sus propias especificaciones (P-01)	El equipo propuso “abstract factory method”; se aclaró que no aplica Abstract Factory (no hay familias de productos relacionadas, solo una: medicamento) y que el nombre correcto del candidato es Factory Method	Corregimos	El Anexo A advierte contra Abstract Factory sin familias reales. Se conservó la intención (una fábrica por presentación) bajo el nombre correcto. Evidencia: nota “Factory Method en P-01” de la Actividad 2.
B-04	Qué patrón resuelve que el contrato de descuento no pueda expresar de quién depende (P-03)	Ninguna — el equipo ya traía evaluados Command y Strategy	Fue idea nuestra	“Pensamos en Command pero da un nivel de complejidad que no es necesaria para el dominio... Strategy es la mejor opción, nos ofrece polimorfismo para diferentes tipos de descuento.” Command descartado, Strategy adoptado.
B-05	Qué patrón conecta el flujo de venta con la notificación, evaluando Bridge, Composite y Observer (P-08)	Ninguna — evaluación propia del equipo sobre el código de un compañero	Fue idea nuestra	“Bridge es cuando las clases evolucionan por separado, y acá todo evoluciona junto... Composite no aplica, no hay que modelar una jerarquía... Observer nos permite que distintos componentes reaccionen cuando sucede una venta.”
B-06	Por qué el proceso de una venta necesita el patrón State y cómo se modelan los estados de la venta (Pendiente, Facturada, Procesada, Confirmada, Fallida)	Ninguna — la decisión ya estaba planteada por el equipo en la implementación	Fue idea nuestra	“Identificamos que siempre que se hacía el proceso de una venta había ciertos pasos obligatorios que dependen uno de otro — se registra la venta, después el total, después la notificación, en ese orden — así que vimos que lo más conveniente era un patrón State.” La bitácora confirma que el cambio se hizo para reemplazar el enum por una jerarquía de estados con comportamiento propio y transiciones controladas por el State.
B-07	Por cuál patrón empezar a implementar, y qué hacer con la fábrica de medicamento que un compañero dejó a medio camino	Cuatro patrones posibles para empezar, y tres formas de tratar el trabajo del compañero (no tocarlo, construir sobre lo que dejó, o esperar)	Aceptamos	Se empezó por Strategy P-02 (no chocaba con lo que el compañero tocaba en paralelo) y se decidió construir sobre su fábrica en vez de ignorarla o pausar el trabajo.
B-08	Qué hacer cuando el commit de un compañero borró el Factory Method de	Restaurar la implementación propia, esperar a hablar con el compañero, o asumir que él traía su propio diseño	Rechazamos las tres	“Lo vamos a dejar como él lo hizo” — el equipo priorizó el trabajo del compañero sobre reabrir un archivo que él seguía modificando, en vez de restaurar o esperar.

ID	Qué consultaron	Qué propuso la herramienta	Qué hicieron	Argumento propio y evidencia
	P-01 que ya se había implementado y verificado			
B-09	Si lo que quedó tras esa reversión (un método Crear con dos sobrecargas) seguía siendo Factory Method	No: el switch seguía vivo decidiendo cuál sobrecarga llamar, quien invoca debe saber cuál firma usar — se reclasificó como no-patrón y se sacó de la matriz SOLID	Aceptamos	Contrastado contra el error típico del enunciado (“una fábrica que crece con un condicional por cada tipo nuevo”): el punto de modificación se movió, no desapareció. Evidencia: RepositorioProductos.cs con el switch sobre datos[0].
B-10	Qué argumento registrar para no mantener el Factory Method que ya estaba implementado y verificado	Dos motivos posibles (P-01 no es la solicitud a demostrar, o prioridad de tiempo/coordinación)	Fue idea nuestra	Se descartaron los dos motivos propuestos y se dio uno propio: “el cumplimiento de SOLID a partir de una única sobrecarga invocable desde un servicio en un solo método; agregar cosméticos, comestibles y otros productos se podrá hacer sin tocar nada existente, cada uno con su propio factory y servicio.” El eje real de extensión (SC-1) no pasa por ese switch.
B-11	— (corrección directa sobre la documentación ya escrita, no una consulta)	Se había descrito la reversión del compañero como si el equipo “no hubiera hecho nada”, sacando la fila por completo de la matriz	Corregimos	“No es que no se haya hecho nada, se implementó una factory normal, si hay un patrón ahí” — la fábrica con sobrecargas es Simple Factory, un patrón real y nombrado, y merece una evaluación SOLID honesta, no un descarte por dado.
B-12	Al rehacer esa evaluación con honestidad, qué encontró la herramienta al auditar el código real del compañero	RepositorioProductos tenía un constructor por defecto que crea su propia FabricaMedicamentos internamente, y Program.cs crea otra instancia distinta — DIP roto, sin compensación real	Aceptamos	Se documentó el hallazgo sin tocar el código —decisión del equipo, no de la herramienta—. Evidencia: matriz de la Actividad 4, fila Simple Factory, columna DIP = Roto.

RETO 2 — ACTIVIDAD 4

Entregable 4.1 — Matriz de verificación

Patrón adoptado	SRP	OCP	LSP	ISP	DIP
Factory — P-01 (medicamento)	Refuerza	Refuerza	Refuerza	Neutro	Refuerza
Strategy — P-02 (puntos)	Refuerza	Refuerza	Neutro	Neutro	Refuerza
Strategy — P-03 (descuento)	Refuerza	Refuerza	Neutro	Neutro	Tensionado pero compensado
Observer — P-08 (notificación y	Refuerza	Refuerza	Neutro	Neutro	Refuerza

Patrón adoptado	SRP	OCP	LSP	ISP	DIP
orquestración de venta)					
State — P-09 (ciclo de vida de la venta)	Refuerza	Refuerza	Refuerza	Neutro	Neutro

Factory — P-01 (medicamento)

SRP — Refuerza. RepositorioProductos.Cargar solo parsea la línea y consulta el registro de creadores; ya no decide relleno/envase ni construye nada. CreadorCapsula/CreadorLiquido solo deciden qué presentación usar al cargar desde archivo y delegan la construcción en IFabricaMedicamentos.

OCP — Refuerza. El switch desapareció de RepositorioProductos. Una presentación nueva es una clase ICreadorMedicamento más una línea en el registro de Program.cs. Verificado: los diez productos de productos.txt cargan igual con el mecanismo nuevo. Límite declarado: el registro resuelve presentaciones de medicamento, no categorías futuras (cosméticos, comestibles de SC-1).

LSP — Refuerza. CreadorCapsula y CreadorLiquido comparten la misma firma de ICreadorMedicamento.Crear y el mismo tipo de retorno; ninguno impone una precondition que el otro no tenga. RepositorioProductos los trata de forma intercambiable.

ISP — Neutro. ICreadorMedicamento es de un solo método, angosta desde el diseño.

DIP — Refuerza. La versión final solo tiene el constructor que recibe el registro, y Program.cs construye una única instancia compartida entre los dos creadores y ServicioProducto.

Strategy — P-02 (regla de puntos)

SRP — Refuerza. IReglaPuntos.Calcular decide cuántos puntos resultan de una base; ServicioPuntos calcula la base desde el total de la venta y coordina; Cliente.AcumularPuntos solo valida y suma.

OCP — Refuerza. ReglaPuntosDoble no toca Cliente.cs ni ServicioPuntos.cs. Verificado: una venta de Dolex × 2 (total 10000) generó automáticamente "Cliente Carlos acumuló 10 puntos", sin teclearlos.

LSP — Neutro. Calcular(int) → int es una función total en ambas implementaciones.

ISP — Neutro. IReglaPuntos es de un solo método.

DIP — Refuerza. ServicioPuntos sigue dependiendo de IReglaPuntos inyectada por constructor; solo cambió el disparador, ahora automático desde EventoVenta.MovimientoRegistrado.

Strategy — P-03 (descuento)

SRP — Refuerza. DescuentoPorcentual/SinDescuento solo calculan un descuento; DescuentoPorcentual ahora valida su propio porcentaje en el constructor.

OCP — Refuerza. Un convenio nuevo es una clase que implementa IDescuento. Verificado: con SinDescuento, una venta de 10000 no tiene descuento, sin tocar ServicioFacturacion.

LSP — Neutro, advertencia resuelta. El constructor de DescuentoPorcentual ahora exige $0 \leq \text{porcentaje} \leq 1$ (antes no validaba). ServicioFacturacion agrega además una guarda sobre el resultado ya calculado — doble validación, sin costo real.

ISP — Neutro. IDescuento es de un solo método.

DIP — Tensionado pero compensado. ServicioFacturacion.CalcularTotal(IFacturable, int, Cliente) y ProcesarVentaSolicitada reciben Cliente/ContextoVenta concretos. Se compensa porque son objetos de dominio, no infraestructura, y la polimorfia real —qué algoritmo corre— la resuelve IDescuento.

Observer — P-08 (notificación y orquestación de venta)

SRP — Refuerza. EventoVenta tiene cuatro señales (VentaSolicitada, FacturaCalculada, VentaProcesada, MovimientoRegistrado) y cada suscriptor reacciona a una sola: ServicioFacturacion factura, ServicioInventario descuenta stock, ServicioMovimiento registra, ServicioPuntos acumula. Ninguno conoce al anterior ni al siguiente.

OCP — Refuerza. Verificado: una venta completa atraviesa las cuatro señales sin que ningún servicio llame a otro directamente. Ya se demostró agregando NotificacionArchivo como segundo suscriptor sin tocar el primero (consola).

LSP — Neutro. Los suscriptores son delegados, no una jerarquía de tipos sustituyéndose.

ISP — Neutro. IServicioNotificacion.EnviaNotificacion(string) es de un solo método.

DIP — Refuerza. Los cuatro servicios dependen de EventoVenta (mecanismo de C#, no negocio); Program.cs es el único lugar que decide qué canal concreto está activo. Verificado: la salida solo difiere de la del sistema sin patrones en lo que las tres solicitudes ya autorizan.

State — P-09 (ciclo de vida de la venta)

SRP — Refuerza. Cada EstadoVenta* tiene una única razón de cambio: qué transiciones acepta. ContextoVenta delega en Estado, no decide nada por sí mismo.

OCP — Refuerza. Un estado nuevo es una clase que hereda de EstadoVentaBase, sin tocar los existentes; cada uno sobrescribe solo lo que le corresponde.

LSP — Refuerza. No cae en la trampa de Template Method que advierte el enunciado ("pasos que alguna subclase no puede cumplir"): EstadoVentaBase no finge cumplir un paso, lanza una excepción con nombre y significado (InvalidOperationException) como resultado legítimo. ContextoVenta llama

Estado.Procesar(this) sin conocer el estado concreto y siempre obtiene un resultado válido. Verificado: vender más unidades de las que hay en stock ya no tumba la aplicación — produce un mensaje controlado.

ISP — Neutro. IEstadoVenta declara cuatro métodos; un estado terminal como EstadoVentaConfirmada no los usa todos activamente, pero es un contrato pequeño y cerrado (las transiciones posibles de una venta), no una implementación forzada.

DIP — Neutro. Cada estado instancia directamente el siguiente (new EstadoVentaFacturada()), la forma estándar de State con un conjunto cerrado y pequeño de estados.

Entregable 4.2 — Evidencia de preservación de comportamiento (resumen)

Mismo funcionamiento, Evidencias en: [Reto2-Actividad4-Entregable42-Evidencias.docx](#)

RETO 2 — ACTIVIDAD 5

Análisis de Riesgos

Entregable 5.1 — Registro de riesgos

ID	Riesgo (si ocurre X, entonces Y)	Prob	Imp	Exp	Qué hacen para evitarlo	Cómo se enteran de que está pasando	Estado al cierre · Exp. residual
R-01	Convenios antes de que la venta conserve el importe: se cobraría un total y el historial mostraría otro.	4	5	20	Orden obligatorio P-04 → P-03.	Los dos totales no coinciden; hoy es el mismo número guardado (ServicioFacturacion.cs:22-53, Program.cs:410).	Cerrado · 5
R-02	Categorías nuevas sobre el ensamblaje roto: cada repositorio copiaría el molde de fabricar por dentro y P-01 volvería por categoría.	4	4	16	Arreglar el ensamblaje antes de escribir la primera categoría.	Un new de concreto fuera de Program.cs; ya no queda ninguno.	Cerrado · 4
R-03	Diagramas terminados antes que el código: las vistas describirían un sistema que ya no existe.	4	4	16	Congelar el código antes de exportar y revisar la correspondencia clase por clase.	Una clase dibujada sin archivo.	Se materializó · 16
R-04	Un canal de avisos tumba la venta: la existencia queda descontada y la venta sin registrar.	3	5	15	Cada suscriptor absorbe su falla y la cadena la convierte en venta fallida (Program.cs:182-195).	El archivo de avisos deja de crecer mientras entran ventas.	Cerrado · 5
R-05	Estrategias de demostración entregadas cableadas: se confundiría una diferencia no autorizada con una autorizada.	3	5	15	Se entrega con ReglaPuntosEstandar y las cuatro diferencias se enumeran en la vista técnica.	La comparación de salidas deja de coincidir con las cuatro diferencias autorizadas.	Vigente y ampliado · 15

ID	Riesgo (si ocurre X, entonces Y)	Prob	Imp	Exp	Qué hacen para evitarlo	Cómo se enteran de que está pasando	Estado al cierre · Exp. residual
R-06	Validaciones que terminan la aplicación: un dato inválido cierra el programa a mitad de operación.	3	4	12	Vender de más valida antes de tocar la existencia; los caminos que abortan quedan declarados como deuda.	Stock descontado sin movimiento.	Parcial · 9
R-07	Porcentaje mal expresado (15 en vez de 0,15): el descuento supera al precio.	3	4	12	Validar el intervalo [0,1] en el constructor; la facturación rechaza descuentos mayores que el subtotal.	Un total ≤ 0 .	Cerrado · 4
R-08	Desbordamiento de alcance al cerrar P-04: el cambio crece más allá de lo acordado.	2	5	10	Dos proyectos, ningún paquete externo, ensamblaje manual.	Una referencia o un proyecto nuevos.	Cerrado · 5
R-09	Convenio asignado a quien no corresponde (nuevo): se cobra bien un descuento que no debía existir y el sistema se comporta con normalidad.	3	5	15	Asignación en un solo sitio y por cédula; sin lista autorizada no se enciende ninguno.	Una cédula con convenio que no esté en la lista autorizada.	Nuevo · 15
R-10	Etapas nuevas después del descuento de existencias (nuevo): la venta termina fallida con el stock ya descontado.	2	4	8	Decidir antes si el descuento se mueve al final o si el estado fallido devuelve lo descontado.	Una venta “no registrada” que baja la existencia.	Nuevo · 8

ACTIVIDAD 6 · Las dos vistas

Vista para el negocio

Para quién. Para la Dirección de Ingeniería y para quien aprueba el presupuesto. No hace falta saber programar para leerla.

Qué debe responder. Qué hicimos: juntamos en un solo sitio cada decisión que estaba escrita en varios —puntos, descuento, forma de armar un producto, a quién se avisa— y, lo único que se nota, la venta ahora calcula lo que se cobra y lo guarda. Qué cambia en el mostrador: solo la pantalla de vender —pide el nombre del cliente, dice cuánto se cobró, ya no se cae al vender de más y acumula puntos sola—; el resto es idéntico y ningún cliente paga distinto. Qué gana la farmacia: un convenio pasa de 4 sitios y hasta 10 partes nuevas a un solo sitio; una promoción, de 3 sitios a una línea; una familia de productos nueva, de 4 sitios a 2. Qué cuesta: tiempo sin función visible, más partes que leer la primera vez, y dos caminos que aún terminan la aplicación. Qué decide el negocio: la lista de entidades con convenio y su porcentaje, lo único que falta para encenderlos.

Qué no puede aparecer. Ni un nombre de clase, archivo, interfaz o patrón; ni rutas, ni líneas, ni jerga técnica. Si al quitarle las palabras técnicas la vista se queda sin contenido, es que no había contenido de negocio.

Prueba de que sirve. Se le presenta a una persona ajena al equipo y sin formación técnica y se anota qué entendió, sin ayudarlo: ¿qué le hicieron al sistema?, ¿qué notará quien atiende en el mostrador?, ¿qué gana la farmacia?, ¿cuál es el riesgo más caro y cómo se darían cuenta?, ¿qué falta para los convenios y de quién depende? La evidencia queda en el video.

Vista para el equipo de desarrollo

Para quién. Para quien entra al equipo dentro de seis meses y tiene que hacer un cambio sin romper nada, sin haber estado en ninguna reunión y sin leerse el sistema entero.

Qué debe responder. Qué patrón hay y dónde vive: Simple Factory (P-01), Strategy de puntos y de descuento (P-02/P-03), Observer (P-08) y State (P-04). Cómo se relacionan: la cadena de la venta —ningún servicio llama a otro; cada uno se suscribe a un evento, mueve el estado del ContextoVenta y dispara el siguiente—. Dónde se ensambla todo: AppFarmaciaConsola/Program.cs, cableado manual, cadena en :95-108 y regla de puntos en :81. Qué reglas no se rompen: comportamiento observable y archivos de datos congelados, ningún new de un concreto fuera del ensamblaje, ningún suscriptor que lance hacia el evento. Qué deuda quedó, con D-03b —nadie asigna todavía un descuento— y D-11 —un fallo tardío deja la existencia descontada— como las que condicionan trabajo futuro.

El artefacto central. La guía de dónde tocar: diez entradas, C-01 a C-10, cada una con lo mismo —qué crear, qué modificar, qué no tocar y cómo verificar—. Las tres solicitudes del Anexo B son C-02 (categorías nuevas), C-03 (servicios) y C-04 (convenios); C-09 y C-10 existen para decir que no se hacen sin autorización.